

Designing a Notification Service: A Complete Derivation Log

Source: An original question from Coupang interview reports (1point3acres), appearing in both the phone screen and the onsite loop — the company’s most frequently reported system design problem.

Original prompt: Design a notification service. Internal users can configure rules; the system sends different types of notifications to different types of external users according to those rules. High concurrency and low latency are required.

This document records the **derivation process itself**, including the approaches that were killed mid-design by arithmetic, and how the architecture converged after each one.

1. Clarification and Scope

The prompt is deliberately underspecified. Three things must be established before designing — this is not a formality, because **the SLO determines the architectural fork**.

1.1 Questions to ask up front

Question	Why it decides the architecture
End-to-end latency requirement?	Seconds → polling suffices; milliseconds → a persistent-connection push channel is mandatory
Maximum fan-out per notification?	The fan-out ceiling decides whether per-user materialisation is even possible
Read/write ratio?	Determines where to spend optimisation effort
Delivery semantics?	at-most-once vs at-least-once; decides whether retries and idempotency are needed

1.2 Two layers of “rules”

The word “rules” in the prompt is ambiguous and must be split, or the answer will address the wrong scope:

- **Generation rules** — which event produces which notification for whom. For instance, an ML model output crossing a threshold, or an order status change. This layer belongs to upstream business systems and the configuration plane.
- **Distribution rules** — once a notification exists, how it is delivered: throttling, frequency capping, deduplication. This layer is the service under design.

Stating this distinction out loud is itself a positive signal, because it demonstrates the candidate understands the system boundary.

1.3 Scope statement

Upstream business systems emit trigger events. Internal users configure rules mapping a trigger to (receiver set, notification type) . This design begins at the output of that mapping and focuses on the downstream distribution path.

The boundary must be spoken. Having it in your head but not saying it leaves the interviewer no choice but to read it as an omission.

2. Data Flow

1. Upstream business systems emit trigger events → rule evaluation
2. Materialise into notification records: receiver_id, notification_type, metadata, timestamp , ordered by time
3. Event-driven: a downstream consumer reads newly produced notifications
4. Deduplicate per user

5. User online → send a signal; user offline → persist and wait for a pull
6. Client fetches on connect or on its polling schedule

Trading product shape for system cost: a notification bell only needs the most recent dozen or so entries, and the client can cache older ones. This single product decision materially reduces read and write pressure — proactively raising product-side trade-offs is a senior signal in system design.

3. Push or Pull: The Fork the SLO Creates

3.1 The fork

This is the first pivotal judgement in the problem: **the end-to-end latency SLO determines the architecture.**

- Loose SLO (seconds acceptable) → client polling. Simple; no connections to maintain
- Strict SLO (milliseconds) → server push. Requires persistent connections, online-state tracking, fan-out orchestration

The two paths differ by an order of magnitude in complexity. Naming this fork and asking the interviewer for the SLO is better than silently picking one.

3.2 Why pure pull is insufficient

Polling is fine for a newsfeed but not for notifications. Payment confirmations and emergency alerts need second-level latency or better. **A polling interval of N seconds means an average latency of N/2**, which is unacceptable for real-time notifications.

3.3 Why pure push does not work either

The cost of push lies in **connections**, not data. For a notification fanning out to a million recipients, pushing to each connection means:

- Maintaining millions of concurrent persistent connections server-side
- Enormous instantaneous bandwidth and connection-management overhead
- Most target users are offline, so the push is wasted

A useful reference point: Amber Alerts can reach everyone because they use carrier-level **cell broadcast** — base stations broadcasting to every device in range — which **never touches an application server**. Ordinary applications have no such capability.

3.4 Final design: signal plus pull

One unified channel, regardless of fan-out size:

1. The server pushes only a **signal** (“you have new messages”), a payload of tens of bytes
2. The client fetches the content itself upon receiving the signal
3. The client also keeps a fallback poll (say, every 10 minutes), so a missed push self-heals
4. The client deduplicates by `notification_id`

Push therefore carries latency, not reliability. A failed push is not retried — the next fallback poll will cover it. This trade-off keeps the entire push path very simple.

4. Persistent Connections and Online State

4.1 Components

- **Connection gateway tier** — each machine holds hundreds of thousands of persistent connections
- **Registry** — `user_id` → `gateway_id`, an in-memory KV with a TTL
- **Heartbeat renewal** — clients heartbeat periodically; a lapsed TTL means offline

4.2 Choosing the heartbeat interval

The interval is governed by how long a “phantom online” window is tolerable — the user has disconnected but the server still believes otherwise, so pushes during that window are lost.

Interval	Phantom window	Write pressure (2M online)
5 s	~15 s	400K QPS
30 s	~90 s	67K QPS
60 s	~180 s	33K QPS

Choose a 30-second heartbeat with a 90-second TTL. Reasoning:

- Every heartbeat wakes the mobile radio; a 5-second cadence is hard on battery, and both iOS and Android throttle it in the background
- The cost of a phantom-online window is very low — a dropped push is covered by the fallback poll; no data is lost
- Write pressure differs by 6×

Gateway-local aggregation: heartbeats land in gateway memory first, and the gateway renews against the registry on a longer cycle (say every 30 seconds). The client keeps a frequent heartbeat (so disconnection is detected quickly) while registry write pressure drops by an order of magnitude.

5. Capacity Estimation

5.1 Assumptions

Item	Value
Registered users	1 billion
DAU	10 million
Peak concurrent online	2 million
Peak notification generation	50,000/s
Retained per user	Most recent 1,000
Per notification	8-byte timestamp + ~100-byte body $\approx$ 108 bytes; 22 bytes compressed
Machines	200

5.2 Storage

```
Per user: 1,000 × 22 bytes      = 22 KB
Total:    1e9 × 22 KB           = 22 TB
Per node: 22 TB ÷ 200           = 110 GB
3 replicas: 66 TB, i.e. 330 GB per node
```

Storage is not the bottleneck. 22 TB across 200 machines is comfortable; write QPS is the binding constraint.

Note that storage must be estimated against the **full registered population**, independent of who is online — notifications are persisted whether or not the recipient is connected. DAU is used only to estimate read QPS.

5.3 Sharding

- Shard key: `user_id` hash
- Roughly **5 million users** per node

- All of a user's notifications live on one node, so a read is a single-shard query
- Eviction: a 1,000-entry cap per user, trimmed on write, keeping total volume flat

5.4 Three fan-out tiers

This is the pivotal design decision of the problem. Start from a proof by contradiction:

If all 50K notifications per second each fanned out to a million recipients, per-user materialisation would mean **50 billion writes per second**, or **1.1 TB/s** at 22 bytes each. Spread over 200 machines that is 5.5 GB/s per node — beyond the sequential-write ceiling of a single NVMe SSD (1–3 GB/s). Scaling to 2,000 machines still leaves 50 million writes per second per node.

Conclusion: this load cannot be materialised per user at any machine count.

Hence the tiering:

Tier	Volume	Fan-out	Path	Storage writes
Regular	40K/s	≤ 100	Write fan-out: route by <code>user_id</code> hash, append to the recipient's inbox; push a signal if online	4M/s at peak
Emergency broadcast	10K/s	Region-wide	Not materialised. Fan out in the connection tier by geographic group; clients register <code>region:xxx</code> on connect	~10K/s (one record only)
Non-urgent large fan-out	Remainder	Large	Not materialised. Store one campaign record plus an audience predicate; merge on the read path when the client pulls	Proportional to campaign count; negligible

Routing predicate: each notification carries `priority` and `estimated_fanout` at ingress and is routed accordingly. The fan-out threshold falls out of the write budget: $200 \text{ nodes} \times 20K \text{ QPS} \div 40K \text{ notifications/s} \approx 100$.

The core trade: the entire write budget goes to tier one; the other two tiers trade “no fan-out” against write amplification, moving the cost to read-path merging and connection-tier fan-out.

5.5 Why a batch pipeline does not rescue large fan-out

A natural instinct is to accumulate and shuffle in batch (MapReduce / Presto / Spark), transposing `notif → user` into `user → notif` and bulk-loading the result. The arithmetic:

```
One minute of accumulation:
  5,000/s × 60 s × 100K fan-out    = 30 billion intermediate rows
  At 22 bytes                      = 660 GB of shuffle data
Write side:
  200 nodes absorbing 150M rows / 3.3 GB each ≈ 55 MB/s — bulk load can do this
```

The write side just about holds, but **storage growth rate** becomes the new bottleneck: 660 GB added per minute, 40 TB per hour, overrunning the 22 TB total in half an hour.

A batch pipeline changes the efficiency of the write path, not the volume of data. The only correct answer for large fan-out remains: do not materialise.

5.6 Per-node resource accounting

Three data structures live on each node:

Data	Distribution	Size per node
user notif table	Sharded by <code>user_id</code> hash	110 GB (5M users × 22 KB)
region notif table	Fully replicated	~300 MB (100K regions × 100 entries × 30 bytes)
<code>user_id</code> → gateway	Sharded by <code>user_id</code> hash	1 MB (2M online ÷ 200 = 10K entries × 100 bytes)

Full replication of the region table means all three sub-requests in a client pull — personal notifications, regional notifications, large-fan-out merge — **land on the same machine**, with no cross-node queries.

5.7 Write and push QPS

Writes:

4M/s peak ÷ 200 nodes = 20K QPS per node (append-only)
 22 bytes per write → 440 KB/s per node — comfortable for both network and disk

Push signals:

Fraction online on a given node = (2M ÷ 200) ÷ 5M = 0.2%
 Push QPS per node = 20K × 0.2% = 40 QPS
 Global = 40 × 200 = 8,000 QPS

This assumes recipients are uniformly distributed across users. In a real system active users receive disproportionately more notifications, so the ratio skews higher — but the order of magnitude holds for estimation.

Reads (pulls):

2M online ÷ 10 s polling interval = 200K QPS
 Per node = 200K ÷ 200 = 1,000 QPS

Each pull resolves and merges three datasets locally.

Egress bandwidth:

200K QPS × ~1 KB average = 200 MB/s
 Per node: 1 MB/s — negligible

Conclusion: read-side pressure (200K QPS) is 25× the push side (8K QPS). This confirms the architectural judgement: **push is a latency optimisation; pull is the primary channel.**

6. Read-Path Merging

A single client pull merges three sources:

1. **Personal inbox** — direct local query by `user_id`
2. **Regional broadcasts** — from the local region table, taking the delta by the user’ s region and `last_read_ts`
3. **Large fan-out content** — evaluate audience predicates for membership and merge at read time

The three are merged by timestamp and deduplicated by `notification_id`.

Offline catch-up: the user table records `last_active_time`. On reconnect the client fetches the delta since that timestamp, subject to a “most recent N” cap so that long-dormant users do not pull excessive history.

Audience evaluation: targeted broadcasts (a city, a segment) store the audience as a **predicate expression** rather than an expanded user list, evaluated against user attributes at pull time. This is the key to avoiding write amplification — storing a predicate costs $O(1)$; expanding a list costs $O(\text{audience size})$.

7. How Industry Solves This

Scenario	Industry approach	Key property
Large fan-out push	Topic subscription. Clients subscribe to a topic; the server publishes once and the push infrastructure fans out (native in FCM/APNs)	Fan-out happens in the connection tier , not the storage tier
Feed distribution	Hybrid fan-out. Write fan-out for ordinary users, read fan-out for high-follower accounts, with a threshold around tens of thousands of followers (Twitter’s classic design)	Route by fan-out magnitude
Marketing notifications	Store predicates, not lists (Braze, Airship and similar). The audience is materialised only when a batch scan runs at send time	Inherently asynchronous and minute-scale; no second-level delivery promise

Mapping back to the three tiers: real-time small fan-out → write fan-out plus push signal; real-time large fan-out → topic/region broadcast with connection-tier fan-out; non-real-time large fan-out → stored predicates and asynchronous batch.

The tiering dimension is the product of «latency requirement × fan-out size», not either one alone.

8. Failure Modes

Failure	Handling
Notification shard unavailable	Replica failover. Writes during the failover window are split by <code>priority</code> : high priority enters a retry queue, low priority is dropped (missing one marketing message is survivable; an order status update is not)
Gateway node loss	Hundreds of thousands of connections drop at once; clients reconnect to other gateways and re-register. Registry TTLs expire the stale entries automatically
Registry unavailable	Push degrades to unavailable and everything falls back to polling. This graceful degradation is possible precisely because push never carried reliability
Push failure or loss	No retry. The client’s 10-minute fallback poll covers it
Duplicate trigger delivery	Message queues are typically at-least-once. Deduplicate on the idempotency key <code>(trigger_id, user_id)</code> , or users see the same notification twice
Consumer backlog	Backpressure and degradation: drop low-priority traffic, defer the non-real-time tier. Queue depth is the earliest warning signal
Data loss	Write-ahead log plus replication. The system is provisioned for peak QPS with headroom

9. Metrics

Business - Delivery rate, click-through rate - Conversion broken down by `notification_type`

System - End-to-end latency P50 / P99 (trigger emitted → device received) - Push success rate - **Empty-pull rate** — if 99% of polls return nothing, the interval can be lengthened, saving money directly - Queue depth (earliest signal of backlog) - Write QPS skew across shards

Capacity - Concurrent connection count - Registry hit rate (phantom-online fraction) - Storage growth rate

10. Design Takeaways

1. **The SLO creates the architectural fork.** Seconds means polling; milliseconds mandates push. Ask before designing.
2. **Separate the two layers of rules.** Generation (who receives what) and distribution (how it is delivered, throttling) are different concerns; state the boundary explicitly.
3. **Push does not carry reliability.** It carries latency only. Failures are not retried; the client's fallback poll guarantees eventual delivery. This trade-off keeps the push path simple and lets the system degrade gracefully when the registry fails.
4. **Fan-out magnitude decides the materialisation strategy.** Small fan-out writes; large fan-out reads. The predicate is the write budget, not the notification type.
5. **Arithmetic kills designs.** $50K \text{ QPS} \times 1M \text{ fan-out} = 50 \text{ billion writes per second}$ — infeasible at any machine count. That single calculation eliminated per-user materialisation, and a batch pipeline does not save it either, because batching changes write efficiency rather than data volume. **Compute before committing.**
6. **Estimate storage against the full population, read QPS against DAU.** The denominators differ; conflating them yields wrong conclusions.
7. **Read-side pressure dominates push-side pressure** — by $25\times$ in this design. That determines where optimisation effort belongs.